

Beginner Friendly Explanation — AI Mother's Day Letter Generator

This document explains the project step-by-step for absolute beginners.

You will understand:

- How the code works
 - What each line does
 - How the UI works
 - How AI is connected
 - Complete code flow
-

What This Project Does

This project creates a beautiful Python application where a user:

1. Enters:
2. Their name
3. Mom's name
4. A special memory
5. Clicks a button
6. AI generates a heartfelt Mother's Day letter automatically.

We use:

- Python
 - Tkinter (for UI)
 - Google Gemini AI API
-

Overall Code Flow

Here's the complete project flow:

```
graph TD; A[User opens app] --> B[Beautiful UI appears];
```

User enters details



Clicks Generate button



Python collects input values



Prompt is created



Gemini AI API is called



AI generates response



Letter appears on screen



Full Code

```
from tkinter import *
from tkinter import messagebox
from google import genai

# ----- AI SETUP ----- #

client = genai.Client(api_key="YOUR_API_KEY")

# ----- FUNCTION ----- #

def generate_letter():

    your_name = your_name_entry.get()
    mom_name = mom_name_entry.get()
    memory = memory_entry.get("1.0", END)

    if not your_name or not mom_name or not memory.strip():
        messagebox.showwarning("Missing Info", Please fill all fields 🌸")
        return

    prompt = f"""
    Write a heartfelt Mother's Day letter
    from {your_name} to {mom_name}.

    Include this memory:
    "{memory}"

    Make it emotional, warm, and genuine.
    Write in 3 short paragraphs.

    Start with:
```

```

Dear {mom_name},
"""

try:
    response = client.models.generate_content(
        model="gemini-1.5-flash",
        contents=prompt
    )

    output_text.delete("1.0", END)
    output_text.insert(END, response.text)

except Exception as e:
    messagebox.showerror("Error", str(e))

# ----- UI ----- #

root = Tk()
root.title("🌸 Mother's Day AI Letter Generator")
root.geometry("700x750")
root.config(bg="#fff0f5")

# ----- TITLE ----- #

title = Label(
    root,
    text="🌸 AI Letter For Mom 🌸",
    font=("Helvetica", 24, "bold"),
    bg="#fff0f5",
    fg="#d63384"
)

title.pack(pady=20)

# ----- FLOWER DECORATION ----- #

flower = Label(
    root,
    text="🌷🌸🌹🌺🌻",
    font=("Helvetica", 22),
    bg="#fff0f5"
)

flower.pack()

# ----- NAME INPUT ----- #

Label(
    root,
    text="Your Name",

```

```

        font=("Helvetica", 14, "bold"),
        bg="#fff0f5"
    ).pack(pady=5)

your_name_entry = Entry(
    root,
    width=40,
    font=("Helvetica", 14)
)

your_name_entry.pack(pady=5)

# ----- MOM NAME ----- #

Label(
    root,
    text="Mom's Name",
    font=("Helvetica", 14, "bold"),
    bg="#fff0f5"
).pack(pady=5)

mom_name_entry = Entry(
    root,
    width=40,
    font=("Helvetica", 14)
)

mom_name_entry.pack(pady=5)

# ----- MEMORY INPUT ----- #

Label(
    root,
    text="Favorite Memory❤️",
    font=("Helvetica", 14, "bold"),
    bg="#fff0f5"
).pack(pady=5)

memory_entry = Text(
    root,
    width=55,
    height=5,
    font=
    ("Helvetica", 12), bd=2
)

memory_entry.pack(pady=10)

# ----- BUTTON ----- #

generate_btn = Button(

```

```

    root,
    text=" ✨ Generate Letter ✨ ",
    font=("Helvetica", 16, "bold"),
    bg="#ff69b4",
    fg="white",
    padx=15,
    pady=10,
    command=generate_letter
)

generate_btn.pack(pady=20)

# ----- OUTPUT ----- #

output_text = Text(
    root,
    width=70,
    height=15,
    font=("Georgia", 13),
    wrap=WORD,
    bg="#ffa5dc",
    fg="#444"
)

output_text.pack(pady=20)

# ----- FOOTER ----- #

footer = Label(
    root,
    text="Made with ❤️ using Python + AI",
    font=("Helvetica", 11),
    bg="#fff0f5",
    fg="gray"
)

footer.pack(pady=10)

root.mainloop()

```



Line-by-Line Explanation

1 Importing Libraries

```
from tkinter import *
```

This imports Tkinter.

Tkinter helps us create:

- windows
- buttons
- labels
- textboxes
- UI applications

Without Tkinter:

- no graphical interface
- only terminal output

```
from tkinter import messagebox
```

This imports popup messages.

Used for:

- warnings
- errors
- alerts

Example:

```
messagebox.showwarning()
```

shows a popup warning window.

```
from google import genai
```

This imports Google Gemini AI SDK.

It helps Python communicate with Gemini AI.

Without this:

- Python cannot send prompts to AI.
-

2 Creating AI Client

```
client = genai.Client(api_key="YOUR_API_KEY")
```

This creates connection with Gemini AI.

Think of API key like:

Password + identity card

Google uses it to:

- identify your app
 - allow API access
 - track usage
-

3 Function Creation

```
def generate_letter():
```

This creates a function.

Functions are reusable blocks of code.

This function runs when user clicks:

Generate Letter Button

4 Getting User Input

```
your_name = your_name_entry.get()
```

This gets text entered in:

Your Name field

Example:

Bhawana

gets stored inside:

your_name

```
mom_name = mom_name_entry.get()
```

Gets mom's name from input field.

```
memory = memory_entry.get("1.0", END)
```

Gets memory text from text box.

Here:

"1.0"

means:

- line 1
- character 0

END

means:

- till end of text.

5 Input Validation

```
if not your_name or not mom_name or not memory.strip():
```


Checks if any field is empty.

```
.strip()
```

removes extra spaces.

```
messagebox.showwarning("Missing Info",Please fill all fields 🌸)
```

Shows warning popup.

```
return
```

Stops function immediately.

Meaning:

- don't continue further
- wait for user input.

6 Creating Prompt

```
prompt = f"""
```

Creates AI instruction.

This is called:

Prompt Engineering

We tell AI:

- what to write
- tone
- style
- structure

```
from {your_name} to {mom_name}
```

This uses:

```
f-string
```

Python replaces variables automatically.

Example:

```
from Bhawana to Mom
```

7 Calling Gemini AI

```
response = client.models.generate_content(
```

This sends request to Gemini AI.

```
model="gemini-1.5-flash"
```

Tells Google which AI model to use.

Why this model?

- beginner friendly
- free-tier friendly
- fast
- good text generation

```
contents=prompt
```

This sends our prompt to AI.

8 Showing AI Output

```
output_text.delete("1.0", END)
```

Clears old text.

Without this:

- previous letters remain visible.

```
output_text.insert(END, response.text)
```

Adds AI generated letter into UI.

9 Error Handling

```
except Exception as e:
```

If something fails:

- internet issue
- API issue
- invalid key

Python catches error safely.

```
messagebox.showerror("Error", str(e))
```

Shows error popup.

10 Creating Main Window

```
root = Tk()
```

Creates main application window.

Everything is placed inside this window.

```
root.title(" 🌸 Mother's Day AI Letter Generator")
```

Sets window title.

```
root.geometry("700x750")
```

Sets window size.

Meaning:

```
Width = 700  
Height = 750
```

```
root.config(bg="#fff0f5")
```

Sets background color.

1 1 Labels

```
Label()
```

Used to display text.

Examples:

- headings
 - titles
 - instructions
-

1 2 Entry Widget

```
Entry()
```

Creates single-line input field.

Used for:

- names
 - short text
-

1 3 Text Widget

```
Text()
```

Creates multi-line textbox.

Used for:

- memory input
 - AI output
-

1 4 Button Widget

```
Button()
```

Creates clickable button.

```
command=generate_letter
```

Very important.

This connects button with function.

Meaning:

```
When button clicked → run generate_letter()
```

1 5 pack()

```
.pack()
```

Places widgets on screen.

Without:

```
.pack()
```

widget won't appear.

mainloop()

```
root.mainloop()
```

Starts application loop.

It keeps app running continuously.

Without this:

- window opens
 - immediately closes.
-



Final Beginner Understanding

This project combines:

Part	Purpose
Tkinter	Beautiful UI
Gemini API	AI generation
Prompt	AI instructions
Function	Runs logic
Button	Triggers action
Text widget	Shows result



Real-World Skills You Learned

By building this project, you learned:

- ✓ Python GUI development
 - ✓ API integration
 - ✓ AI application building
 - ✓ Prompt engineering
 - ✓ Error handling
 - ✓ User input handling
 - ✓ Beginner app architecture
-



Future Improvements

You can extend this project with:

- Voice generation
 - Background music
 - PDF export
 - Streamlit web app
 - WhatsApp integration
 - Image generation
 - Email sending
 - AI voice narration
-



Final Message

This is not just a Python project.

It teaches how modern AI applications are built in real-world development.

Small beginner projects like this are the first step toward:

- AI engineering
- Full-stack AI apps
- Real-world automation
- Portfolio projects
- Content creation projects

Keep building 🌸❤️